Documentation of the adapted Fractions Skill Score: Summed Area Table implementation (`fss_SAT.py`)

Phillip Scheffknecht

June 22, 2026

Abstract

This document describes `fss_SAT.py`, the summed area table (SAT) implementation of the Fractions Skill Score (FSS). The implementation is based on the reference code published by Faggian et al. (2015). We first recap the SAT algorithm and its reference implementation, and then document the adaptations and extensions made on top of it: a vectorised neighbourhood filter, a BLAS-based score evaluation, additional threshold modes, percentile thresholds, an over-estimation diagnostic, an ensemble (eFSS) variant, parallelisation over thresholds, transparent handling of missing (NaN) data through per-window valid-point weighting, and the continuous/weighted FSS (`CWFSS`). A final section documents the automated cross-method test suite that guards these adaptations.

Contents

1 Introduction **2**
1.1 Scope of this document . . . 2

2 The summed area table baseline **3**
2.1 Summed area table and windowed sums . . . 3
2.2 Boundary conditions . . . 3
2.3 From windowed sums to the score . . . 3

3 Adaptations in `fss_SAT.py` **5**
3.1 Vectorised neighbourhood filter . . . 5
3.2 BLAS dot-product score . . . 5
3.3 Threshold modes and percentile thresholds . . . 6
3.4 Over-estimation diagnostic . . . 6
3.5 Ensemble FSS (eFSS) . . . 7
3.6 Threshold sweep and parallelisation . . . 7
3.7 Missing-data handling via per-window valid-point weighting . . . 7
3.8 Continuous/weighted FSS (`CWFSS`) . . . 8

4 Verification and test suite **9**
4.1 Coverage . . . 9
4.2 The missing-data tests . . . 9
4.3 Result . . . 10

1 Introduction

The Fractions Skill Score (FSS) is a neighbourhood-based verification metric for spatial forecasts, introduced by Roberts and Lean (2008). Rather than comparing forecast and observation values point by point, it compares the *fraction* of grid points that exceed a given threshold within a sliding window (neighbourhood), as a function of both the threshold and the window (neighbourhood) size. This avoids the *double penalty* that afflicts point-wise scores when a feature is forecast with a small spatial displacement and reveals the spatial scale at which a forecast attains useful skill. It does so by treating the spatial distribution of events within a window probabilistically: a perfect forecast is one with the same event frequency as the observation within the window, regardless of the exact placement.

For a single threshold and window size the score is (Roberts and Lean, 2008)

$$\mathrm{FSS} = 1 - \frac{\frac{1}{N}\sum_{i=1}^{N}\left(p_f - p_o\right)^2}{\frac{1}{N}\sum_{i=1}^{N} p_f^2 + \frac{1}{N}\sum_{i=1}^{N} p_o^2}, \tag{1}$$

where N is the number of windows in the domain (one centred on each grid point), p_f is the forecast fraction and p_o the observed fraction of the window. $\mathrm{FSS} = 1$ denotes a perfect forecast and $\mathrm{FSS} = 0$ no skill.

The cost of the FSS comes from evaluating the windowed fractions p_f, p_o for many window sizes and thresholds. Faggian et al. (2015) showed that this windowed summation can be computed very efficiently using a *summed area table* (also called an integral image): the table is built once per threshold with two cumulative sums, after which the sum over any rectangular window is obtained from just four array lookups, independent of the window size. This is the algorithm implemented in `fss_SAT.py`.

1.1 Scope of this document

`fss_SAT.py` started from the reference Python implementation in the appendix of Faggian et al. (2015). The purpose of this document is to record the *adaptations and extensions* that were made on top of that baseline. It is organised as follows:

- Section 2 recaps the SAT algorithm of Faggian et al. (2015) and its reference implementation, establishing the notation and the starting point for the changes.
- Section 3 documents each adaptation made in `fss_SAT.py`: the performance rewrites that preserve the original result bit-for-bit, the feature extensions (threshold modes, percentile thresholds, ensemble FSS, parallelisation), and the substantive algorithmic additions (missing-data handling and the continuous/weighted FSS).

A companion FFT-based implementation (`fss_FFT.py`) and an OpenCL implementation (`fss_OCL.py`) exist in the same repository and compute the same score; they are out of scope here and documented separately.

2 The summed area table baseline

This section recaps the algorithm of Faggian et al. (2015) as far as it is needed to follow the adaptations in Section 3. The notation follows the paper.

2.1 Summed area table and windowed sums

A field is first reduced to a binary event field I by thresholding (for the classic FSS, $I(i,j) = 1$ where the field exceeds the threshold and 0 otherwise). The summed area table $\hat{O}$ stores at each point the sum of all cells above and to the left, inclusive:

$$\hat{O}(i,j) = \sum_{\hat{\imath} \le i} \sum_{\hat{\jmath} \le j} O(\hat{\imath}, \hat{\jmath}). \tag{2}$$

In the code this is a double cumulative sum (`compute_integral_table`):

```
def compute_integral_table(field):
    if field.ndim == 2:
        return field.cumsum(1).cumsum(0)
    elif field.ndim == 3:
        return field.cumsum(2).cumsum(1)
```

Once $\hat{O}$ is available, the sum over a window of half-width w centred on (i,j) is obtained from just four lookups,

$$\sum_{\hat{\imath}=i-w}^{i+w} \sum_{\hat{\jmath}=j-w}^{j+w} O(\hat{\imath}, \hat{\jmath}) = \hat{O}(i+w, j+w) + \hat{O}(i-w, j-w) - \hat{O}(i-w, j+w) - \hat{O}(i+w, j-w), \tag{3}$$

independent of the window size. This is the key property that makes the SAT method cheap when many window sizes are evaluated per threshold: the table is built once, and every window size is then four array lookups per grid point. In the code the windowed sum is `integral_filter`.

2.2 Boundary conditions

Windows centred near the edge of the domain extend beyond the indexable region of the table. Faggian et al. (2015) clip the corner coordinates of Eq. (3) to the valid index range, which is how the reference implementation (and `fss_SAT.py`) handle the boundary. The reference code realises the clip with `np.mgrid` and `np.clip`:

```
r, c = np.mgrid[0:rows, 0:cols]
r0, c0 = (np.clip(r - w, 0, rows - 1), np.clip(c - w, 0, cols - 1))
r1, c1 = (np.clip(r + w, 0, rows - 1), np.clip(c + w, 0, cols - 1))
integral_table  = field[..., r1, c1] + field[..., r0, c0]
integral_table -= field[..., r0, c1] + field[..., r1, c0]
```

2.3 From windowed sums to the score

Dividing each windowed sum by the window area $(2w+1)^2$ gives the fractions p_f and p_o of Eq. (1). The reference `fss` routine forms the fractions explicitly and evaluates the numerator and denominator with `np.nanmean`:

```
inv_window_area = 1. / (2. * w + 1.) ** 2
fhat = fhat * inv_window_area
ohat = ohat * inv_window_area
num   = np.nanmean(np.power(fhat - ohat, 2))
denom = np.nanmean(np.power(fhat, 2) + np.power(ohat, 2))
ret = num, denom, 1. - num / denom
```

The higher-level routines (`fss_threshold`, `fss_cumsum_*`, `CWFSS`) build the binary tables once per threshold and call `fss`/`integral_filter` for every requested window. This is the structure that Section 3 modifies.

3 Adaptations in `fss_SAT.py`

The changes fall into three groups: performance rewrites that leave the result of Faggian et al. (2015) unchanged (Sections 3.1–3.2), feature extensions (Sections 3.3–3.6), and algorithmic additions that go beyond the original method (Sections 3.7–3.8). Table 1 gives an overview.

Where	Change	Effect
`integral_filter`	`np.pad` + slicing	same result, faster, no fancy indexing
`_fss_score`	BLAS dot products	same result, no temporary arrays
`_build_binary_sat`	threshold modes	`over/under/between/tolerance`
`fss_threshold`	percentile thresholds	threshold given as a percentile of the field
`fss_threshold`	over-estimation `ovest`	frequency-bias diagnostic returned with FSS
`fss_threshold_eps`	ensemble FSS	3D forecast stack, exceedance probability
`fss_cumsum_parallel`	`joblib`	threshold sweep optionally parallel (`n_jobs`)
`_fss_score_masked`	per-window valid weighting	transparent missing-data (NaN) handling
`CWFSS`	R2 sampling, bootstrap, NaN	continuous/weighted FSS

Table 1: Overview of the adaptations made on top of the Faggian et al. (2015) reference implementation.

3.1 Vectorised neighbourhood filter

The reference `integral_filter` (Section 2) builds two full coordinate grids with `np.mgrid`, clips them, and gathers the four corner tables with fancy indexing. The rewrite achieves the identical clamped box-sum of Eq. (3) with edge-replicated padding and plain contiguous slicing, which avoids the large index arrays and the gather:

```
w = n // 2
if w < 1:
    return field
rows, cols = field.shape[-2], field.shape[-1]
D = 2 * w
p = np.pad(field, w, mode='edge')           # 2D case
return (p[D:D+rows, D:D+cols] + p[:rows, :cols]
      - p[:rows, D:D+cols] - p[D:D+rows, :cols])
```

Padding the integral table with `mode='edge'` replicates the boundary rows and columns, so a corner lookup that would have been clamped to the last valid index instead reads the replicated edge value — exactly what the original `np.clip` produced. The boundary behaviour is therefore *identical* to the reference implementation; only the data access pattern changes. The 3D (ensemble) case pads only the two trailing spatial axes.

3.2 BLAS dot-product score

The reference `fss` scales both windowed fields to fractions and then forms two temporary arrays, `(fhat-ohat)**2` and `fhat**2+ohat**2`, before averaging. The rewrite (`_fss_score`) expresses the same quantities through three inner products of the *unscaled* box sums S_f, S_o, which BLAS evaluates without allocating temporaries:

$$ff = \sum_i S_f^2, \qquad oo = \sum_i S_o^2, \qquad fo = \sum_i S_f S_o. \tag{4}$$

Using the identity $\sum_i (S_f - S_o)^2 = ff + oo - 2\,fo$ and writing $A = (2w+1)^2$ for the window area, the numerator and denominator of Eq. (1) become

$$\text{num} = \frac{1}{A^2}\,\frac{ff + oo - 2\,fo}{N}, \qquad \text{denom} = \frac{1}{A^2}\,\frac{ff + oo}{N}, \tag{5}$$

where A^{-2} is the constant `inv_area_sq` factor (the fractions $p = S/A$ enter squared). This is algebraically identical to the reference result, but the score reduces to three `np.dot` calls plus scalar arithmetic:

```
ff = np.dot(fflat, fflat); oo = np.dot(oflat, oflat); fo = np.dot(fflat, oflat)
num   = inv_area_sq * (ff + oo - 2.0 * fo) / n
denom = inv_area_sq * (ff + oo) / n
if denom == 0.0:
    return 0.0, 0.0, np.nan
return num, denom, 1.0 - num / denom
```

Note that `np.dot` is not NaN-aware, unlike the reference `np.nanmean`. This fast path is therefore only taken when the fields contain no missing data; the missing-data case is handled separately (Section 3.7).

3.3 Threshold modes and percentile thresholds

The reference implementation binarises with a single "over" comparison (field $> t$). `fss_SAT.py` factors the binarisation into `_build_binary_sat` and supports four `threshold_mode` values:

`over` field $> t$ (default, the classic FSS);

`under` field $\leq t$;

`between` $t_1 <$ field $\leq t_2$, for verification of a value band (in the sweep, an extra lower edge -1 is inserted so that the lowest band still captures the zero values);

`tolerance` $(1-\tau)\,t <$ field $\leq (1+\tau)\,t$, a relative tolerance band of half-width τ (`tolerance`) around t.

Independently, `percentiles=True` interprets the threshold as a percentile of the field rather than an absolute value; the forecast and observation thresholds are then derived separately with `np.percentile` (`np.nanpercentile` on the missing-data path).

3.4 Over-estimation diagnostic

Alongside the numerator, denominator and score, `fss_threshold` now returns an over-estimation (frequency-bias) value

$$\text{ovest} = \frac{\#\{\text{fcst} > t\} - \#\{\text{obs} > t\}}{N_{\text{points}}}, \tag{6}$$

the difference between the forecast and observed event counts normalised by the number of points. It is a domain-wide bias indicator and is broadcast to the shape of the window axis so it travels through the same return structure as the score. On the missing-data path it is computed over valid points only.

3.5 Ensemble FSS (eFSS)

`fss_threshold_eps` implements the ensemble extension of the FSS in the spirit of Duc et al. (2013). The forecast is a 3D stack (members × rows × cols). Instead of a binary forecast field, the per-point *exceedance probability* across members is used,

$$p_f(i,j) = \frac{1}{M}\sum_{m=1}^{M} 1[\,\mathrm{fcst}_m(i,j) \text{ exceeds } t\,], \tag{7}$$

(M = number of members), and this probability field is fed into the same SAT machinery as the deterministic case. The observation remains a single 2D field. An assertion enforces the 3D forecast shape.

3.6 Threshold sweep and parallelisation

`fss_cumsum_parallel` sweeps a list of thresholds, calling `fss_threshold` (or `fss_threshold_eps` when `eps=True`) once per threshold and reusing the cached integral tables across all windows. The sweep can run serially (`n_jobs=1`) or in parallel over thresholds through `joblib` (`n_jobs>1`):

```
if n_jobs == 1:
    ret = [use_fss_threshold_func(fcst, obs, t1, t2, windows, ...) for t1, t2 in
       calls]
else:
    from joblib import Parallel, delayed
    ret = Parallel(n_jobs=n_jobs)(
        delayed(use_fss_threshold_func)(fcst, obs, t1, t2, windows, ...) for t1,
           t2 in calls)
```

`fss_cumsum_frame` wraps the sweep and returns pandas `DataFrame`s of numerator, denominator, score and over-estimation (indexed by threshold, columns by window), or just the raw score array when `raw=True`. Note that, because BLAS itself is multi-threaded, oversubscription can occur when `n_jobs>1`; the benchmark `bench_njobs.py` measures this with and without pinning the BLAS thread count.

3.7 Missing-data handling via per-window valid-point weighting

This is the most substantial addition. The reference method has no concept of missing data; a single NaN would propagate through the cumulative sums and poison the whole table. `fss_SAT.py` adds a mask-aware path that treats a grid point as *missing* when either the forecast *or* the observation is NaN there (`_validity_mask`). The clean path is taken whenever the mask is all-valid, so fields without NaNs are unaffected and pay no extra cost.

When NaNs are present, the binary fields are zeroed at missing points before the SAT is built (`_build_binary_sat_masked`), so missing points never contribute to a window sum. Each window is then weighted by its number of *valid* points,

$$C = A - (\text{missing points in the window}), \tag{8}$$

where $A = (2w+1)^2$ is the full window area and the missing-point count itself is obtained from a SAT of the invalid mask (`_invalid_sat`) sampled with the same `integral_filter`. With S_f, S_o the box sums

of the mask-zeroed binary fields, the windowed fractions are $p_f = S_f/C$, $p_o = S_o/C$, and weighting each window centre by C collapses the weighted means to

$$\text{num} = \frac{\sum_w (S_f - S_o)^2/C}{\sum_w C}, \qquad \text{denom} = \frac{\sum_w (S_f^2 + S_o^2)/C}{\sum_w C} \tag{9}$$

(`_fss_score_masked`). Windows with no valid points ($C = 0$) drop out naturally. The construction is deliberately consistent with the clean path: with no missing data $C \equiv A$ everywhere, and Eq. (9) reduces *bit-for-bit* to the standard score of Section 3.2. Percentile thresholds on this path use `np.nanpercentile`.

The same weighting is wired into the ensemble path (`fss_threshold_eps`, Section 3.5): there a grid point counts when the observation is valid *and* at least one member is valid, and the exceedance probability is averaged over the valid members only (`_eps_prob_masked`).

3.8 Continuous/weighted FSS (CWFSS)

`CWFSS` aggregates the FSS over a *range* of thresholds and windows into a single scalar, instead of reporting the full threshold × window table. It draws `nsamples` (threshold, window) pairs from the low-discrepancy R2 sequence (`R2`), which gives even coverage of the 2D parameter space without a fixed grid, and computes the FSS for each pair. Each sample is weighted by a threshold factor and a window factor,

$$\text{t\_factor} = \frac{t_{\max} + t}{t_{\max}}, \qquad \text{w\_factor} = \frac{2\, w_{\max}}{w_{\max} + w}, \tag{10}$$

emphasising higher thresholds and smaller windows, and the weighted scores are normalised by their achievable maximum to yield `self.cwfss`. A `bootstrap` method resamples the stored per-sample scores to obtain a confidence distribution.

`CWFSS` carries the same extensions as the batch functions. It honours `threshold_mode`/`tolerance` through a shared `_binarise` helper, and it is NaN-aware: threshold limits use `np.nanmax`/`np.nanpercentile`, and when the fields contain NaNs it uses the masked score of Eq. (9). Samples whose FSS is undefined (NaN, e.g. a window with no valid exceedance) are dropped from the `np.nanmean`, and the theoretical maximum is masked identically so the achievable ceiling stays at 1 and is not lowered merely by the presence of missing data.

4 Verification and test suite

The adaptations of Section 3 are guarded by an automated `pytest` suite, `test_fss_consistency.py`. It serves two purposes at once: it is a *regression* suite that pins the behaviour of the SAT code documented here, and a *cross-method consistency* suite that checks the SAT implementation against the two sibling backends in the repository — the FFT reference (`fss_FFT.py`) and the OpenCL implementation (`fss_OCL.py`). Agreement between three independent code paths that compute the same score is a strong correctness signal, and it is what lets the performance rewrites of Sections 3.1–3.2 be trusted to leave the result unchanged.

All tests run on a small set of reproducible Gaussian-bell fields built from a fixed seed, so every value is deterministic. Two tolerances are used when comparing methods, reflecting genuine algorithmic differences rather than bugs:

10^{-5} between SAT and OpenCL — the *same* algorithm, the only difference being `float32` arithmetic on the device versus `float64` on the host;

10^{-3} between FFT and SAT — the FFT convolution zero-pads at the domain boundary whereas the SAT filter edge-clamps (Section 3.1), so the two agree in the interior but differ in a boundary frame.

The suite is run with

```
pytest test_fss_consistency.py -v
```

and requires no arguments. The OpenCL tests detect whether a `pyopencl` runtime is available and *skip* themselves when it is not, so the suite passes on a machine without a GPU/OpenCL stack.

4.1 Coverage

Table 2 groups the checks by theme. The right-hand column records which of the three backends each theme exercises (S = SAT, F = FFT, O = OpenCL).

4.2 The missing-data tests

Because the missing-data handling of Section 3.7 is the most substantial addition, it carries the most checks, and they are mirrored across all three backends (deterministic and ensemble for SAT and FFT; deterministic for OpenCL, which has no ensemble entry point). The key guarantees are:

Reduction to the clean path. On NaN-free fields the masked score of Eq. (9) is compared against the ordinary fast path. For SAT and FFT this matches to machine precision ($\sim 10^{-9}$ or better), confirming that the valid count C of Eq. (8) collapses to the constant window area A when nothing is missing; for OpenCL it matches to the `float32` tolerance 10^{-5}. A companion test verifies directly that $C \equiv A$ everywhere in the absence of NaNs.

Invariants under NaN. With NaNs injected into both fields, the score stays in $[0, 1]$, a perfect forecast (sharing the observation's NaNs) still scores 1, the forecast/observation symmetry holds, and an all-missing field yields NaN rather than a spurious number.

Weighting semantics. A hand-rolled weighted mean reproduces `_fss_score_masked`, pinning the per-window valid-point weighting of Eq. (9).

Theme	What is verified	Backends
Cross-method	FFT, SAT and OpenCL scores agree within tolerance across thresholds × windows	S,F,O
Mathematical properties	perfect forecast → 1, FSS ∈ [0, 1], NaN when no exceedances, monotonicity in window size, forecast/observation symmetry	S,F,O
Batch interfaces	`fss_threshold`, `fss_cumsum_parallel` and `fss_opencl_arr` match the per-call single results	S,O
Regression guards	pinned reference scores for fixed (threshold, window) pairs, including every `threshold_mode`	S
Threshold modes	`over`/`under`/`between`/`tolerance` preserve invariants and partition the domain correctly; the tolerance window is built from the forecast's own threshold on both bounds	S
`CWFSS`	R2 sequence, numerators / denominators / values / score and `bootstrap` are bit-identical to the reference, plus invariants and pinned values	S
Missing data	the masked path reduces *exactly* to the clean path without NaNs, and preserves all invariants under injected NaNs (see Section 4.2)	S,F,O

Table 2: Themes covered by `test_fss_consistency.py` and the backends each one exercises (S = SAT, F = FFT, O = OpenCL).

Ensemble specifics. A member that is entirely NaN does not change the score (the average is over valid members only), and the `nanpercentile` threshold path stays finite.

Cross-method agreement. The FFT and OpenCL masked paths are checked against the SAT masked path. OpenCL also edge-clamps, so it agrees with SAT as closely as on clean data; FFT zero-pads, so it agrees within the interior at the usual 10^{-3} boundary tolerance.

Auto-dispatch. Calling the public entry points (`fss_threshold`, `fourier_fss`, `fourier_fss_eps`, `fss_opencl`) on a field that contains a NaN routes to the masked path automatically — important for the FFT, where a raw NaN passed through `fftconvolve` would otherwise spread across the entire output.

4.3 Result

The suite currently comprises **308 tests**. With an OpenCL runtime present all **308 pass**; on a machine without OpenCL the **60** device-backed tests skip and the remaining **248 pass**. A representative run completes in roughly six seconds.

References

Faggian, N., Roux, B., Steinle, P. and Ebert, B., 2015: *Fast calculation of the fractions skill score.* MAUSAM, **66**, 3, 457–466.

Roberts, N. M. and Lean, H. W., 2008: *Scale-selective verification of rainfall accumulations from high-resolution forecasts of convective events.* Mon. Wea. Rev., **136**, 78–97.

Duc, L., Saito, K. and Seko, H., 2013: *Spatial-temporal fractions verification for high-resolution ensemble forecasts.* Tellus A, **65**.